# Vision

Learn how to use GPT-4 to understand images

## Introduction

GPT-4 with Vision, sometimes referred to as GPT-4V or `gpt-4-vision-preview` in the API, allows the model to take in images and answer questions about them. Historically, language model systems have been limited by taking in a single input modality, text. For many use cases, this constrained the areas where models like GPT-4 could be used.

GPT-4 with vision is currently available to all developers who have access to GPT-4 via the `gpt-4-vision-preview` model and the Chat Completions API which has been updated to support image inputs. Note that the Assistants API does not currently support image inputs.

It is important to note the following:

- GPT-4 Turbo with vision may behave slightly differently than GPT-4 Turbo, due to a system message we automatically insert into the conversation
- GPT-4 Turbo with vision is the same as the GPT-4 Turbo preview model and performs equally as well on text tasks but has vision capabilities added
- Vision is just one of many capabilities the model has

> Currently, GPT-4 Turbo with vision does not support the **message.name** parameter, **functions/tools**, **response_format** parameter, and we currently set a low **max_tokens** default which you can override.

## Quick start

Images are made available to the model in two main ways: by passing a link to the image or by passing the base64 encoded image directly in the request. Images can be passed in the `user`, `system` and `assistant` messages. Currently we don't support images in the first `system` message but this may change in the future.

```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
  model="gpt-4-vision-preview",
  messages=[
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "What’s in this image?"},
        {
          "type": "image_url",
          "image_url": {
            "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-w
          },
        },
      ],
    }
  ],
  max_tokens=300,
)

print(response.choices[0])
```

The model is best at answering general questions about what is present in the images. While it does understand the relationship between objects in images, it is not yet optimized to answer detailed questions about the location of certain objects in an image. For example, you can ask it what color a car is or what some ideas for dinner might be based on what is in you fridge, but if you show it an image of a room and ask it where the chair is, it may not answer the question correctly.

It is important to keep in mind the limitations of the model as you explore what use-cases visual understanding can be applied to.

**Video understanding with vision**

Learn how to use use GPT-4 with Vision to understand videos in the OpenAI Cookbook

## Uploading base 64 encoded images

If you have an image or set of images locally, you can pass those to the model in base 64 encoded format, here is an example of this in action:

```python

# OpenAI API Key
api_key = "YOUR_OPENAI_API_KEY"

# Function to encode the image
def encode_image(image_path):
  with open(image_path, "rb") as image_file:
    return base64.b64encode(image_file.read()).decode('utf-8')

# Path to your image
image_path = "path_to_your_image.jpg"

# Getting the base64 string
base64_image = encode_image(image_path)

headers = {
  "Content-Type": "application/json",
  "Authorization": f"Bearer {api_key}"
}

payload = {
  "model": "gpt-4-vision-preview",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "What’s in this image?"
        },
        {
          "type": "image_url",
          "image_url": {
            "url": f"{base64_image}"
          }
        }
      ]
    }
```

```
}

response = requests.post("https://api.openai.com/v1/chat/completions", h

print(response.json())
```

## Multiple image inputs

The Chat Completions API is capable of taking in and processing multiple image inputs in both base64 encoded format or as an image URL. The model will process each image and use the information from all of them to answer the question.

Multiple image inputs python Copy

```python
from openai import OpenAI

client = OpenAI()
response = client.chat.completions.create(
  model="gpt-4-vision-preview",
  messages=[
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "What are in these images? Is there any difference between them?"
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-w
          },
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-w
          },
        },
      ],
    }
```

```
print(response.choices[0])
```

Here the model is shown two copies of the same image and can answer questions about both or each of the images independently.

## Low or high fidelity image understanding

By controlling the `detail` parameter, which has three options, `low`, `high`, or `auto`, you have control over how the model processes the image and generates its textual understanding. By default, the model will use the `auto` setting which will look at the image input size and decide if it should use the `low` or `high` setting.

> `low` will disable the "high res" model. The model will receive a low-res 512px x 512px version of the image, and represent the image with a budget of 65 tokens. This allows the API to return faster responses and consume fewer input tokens for use cases that do not require high detail.
>
> `high` will enable "high res" mode, which first allows the model to see the low res image and then creates detailed crops of input images as 512px squares based on the input image size. Each of the detailed crops uses twice the token budget (65 tokens) for a total of 129 tokens.

Choosing the detail level python Copy

```python
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
  model="gpt-4-vision-preview",
  messages=[
    {
      "role": "user",
      "content": [
        {"type": "text", "text": "What's in this image?"},
        {
          "type": "image_url",
          "image_url": {
            "url": "https://upload.wikimedia.org/wikipedia/commons/thumb/d/dd/Gfp-w
            "detail": "high"
          },
        },
```

```
  max_tokens=300,
)

print(response.choices[0].message.content)
```

## Managing images

The Chat Completions API, unlike the Assistants API, is not stateful. That means you have to manage the messages (including images) you pass to the model yourself. If you want to pass the same image to the model multiple times, you will have to pass the image each time you make a request to the API.

For long running conversations, we suggest passing images via URL's instead of base64. The latency of the model can also be improved by downsizing your images ahead of time to be less than the maximum size they are expected them to be. For low res mode, we expect a 512px x 512px image. For high res mode, the short side of the image should be less than 768px and the long side should be less than 2,000px.

After an image has been processed by the model, it is deleted from OpenAI servers and not retained. We do not use data uploaded via the OpenAI API to train our models.

## Limitations

While GPT-4 with vision is powerful and can be used in many situations, it is important to understand the limitations of the model. Here are some of the limitations we are aware of:

- Medical images: The model is not suitable for interpreting specialized medical images like CT scans and shouldn't be used for medical advice.
- Non-English: The model may not perform optimally when handling images with text of non-Latin alphabets, such as Japanese or Korean.
- Small text: Enlarge text within the image to improve readability, but avoid cropping important details.
- Rotation: The model may misinterpret rotated / upside-down text or images.
- Visual elements: The model may struggle to understand graphs or text where colors or styles like solid, dashed, or dotted lines vary.
- Spatial reasoning: The model struggles with tasks requiring precise spatial localization, such as identifying chess positions.